

Resolución del Práctico 4

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Ejercicio 1: grafo de 4 nodos	2
1.1. (a) NC pero no EC	2
1.2. (b) EC pero no EPC	2
1.3. (c) Conjunto mínimo para EPC	3
2. Ejercicio 2: grafo de 7 nodos, <i>sidetrips</i> y PPC	3
2.1. (a) Requisitos para EPC	3
2.2. (b) ¿El conjunto candidato satisface EPC?	3
2.3. (c) Tour directo vs con <i>sidetrip</i>	4
2.4. (d) Requisitos para NC, EC y PPC	4
2.5. (e) NC sin EC	5
2.6. (f) EC sin PPC	5
3. Ejercicio 3: CFG de <code>fmtRerwrap</code>	5
3.1. CFG por bloques	5
3.2. (a) CFG del método	6
3.3. (b) Test que sale del <code>while</code> sin entrar al cuerpo	6
3.4. (c) Requisitos para NC, EC y PPC	6
3.5. (d) Suite NC pero no EC	6
3.6. (e) Suite EC pero no PPC	7
3.7. (f) Suite PPC con <i>Best Effort Touring</i>	8
4. Ejercicio 4: instrumentación de <code>patternIndex</code>	9
4.1. Instrumentación	9
4.2. Suite	10
4.3. Resultados de cobertura	10
4.4. Cómo ejecutar	11

1. Ejercicio 1: grafo de 4 nodos

Grafo dirigido:

- $N = \{1, 2, 3, 4\}$, $N_0 = \{1\}$, $N_f = \{4\}$.
- $E = \{(1, 2), (2, 3), (3, 2), (2, 4)\}$.

Estructura. Desde 1 solo se puede ir a 2; desde 2 se ramifica a 3 o a 4; desde 3 se vuelve a 2; 4 es el único nodo final y no tiene aristas salientes. Por consiguiente, todo camino de test válido tiene la forma:

$$1 \rightarrow 2 \rightarrow \underbrace{(\text{ciclo } 2 \leftrightarrow 3 \text{ repetido } k \geq 0 \text{ veces})}_{\text{opcional}} \rightarrow 4.$$

Caminos válidos posibles: $[1, 2, 4]$, $[1, 2, 3, 2, 4]$, $[1, 2, 3, 2, 3, 2, 4]$, etc.

Requisitos por criterio:

- **NC:** $\{1, 2, 3, 4\}$.
- **EC:** $\{(1, 2), (2, 3), (3, 2), (2, 4)\}$.
- **EPC** (caminos de longitud 2 alcanzables): $[1, 2, 3]$, $[1, 2, 4]$, $[2, 3, 2]$, $[3, 2, 3]$, $[3, 2, 4]$.

1.1. (a) NC pero no EC

No es posible. Cumplir NC obliga a visitar los cuatro nodos, y dada la topología cada visita fuerza un arco específico:

1. Visitar 1 obliga a usar $(1, 2)$ (única arista saliente de 1).
2. Visitar 3 obliga a usar $(2, 3)$ (única arista que entra a 3).
3. Desde 3 solo se puede salir por $(3, 2)$.
4. Terminar en 4 obliga a usar $(2, 4)$.

Esos cuatro arcos son exactamente EC, así que cualquier suite que cumpla NC también cumple EC.

1.2. (b) EC pero no EPC

Sí, alcanza con un solo test:

$$t_1 = [1, 2, 3, 2, 4].$$

Recorre los cuatro arcos del grafo (EC queda cubierta). Los pares cubiertos son $[1, 2, 3]$, $[2, 3, 2]$, $[3, 2, 4]$. Quedan sin cubrir al menos $[1, 2, 4]$ y $[3, 2, 3]$, así que EPC no se satisface.

1.3. (c) Conjunto mínimo para EPC

Dos tests alcanzan:

$$t_1 = [1, 2, 4], \quad t_2 = [1, 2, 3, 2, 3, 2, 4].$$

t_1 cubre $[1, 2, 4]$; t_2 cubre $[1, 2, 3]$, $[2, 3, 2]$, $[3, 2, 3]$ y $[3, 2, 4]$. La unión cubre los cinco pares alcanzables.

Por qué no alcanza con uno solo:

- Para cubrir $[1, 2, 4]$ el test tiene que ir de 2 directo a 4.
- Para cubrir $[1, 2, 3]$ el test tiene que ir de 2 a 3.
- Ambas cosas son incompatibles en el mismo prefijo 1, 2, y una vez en 4 no hay vuelta atrás.

Hacen falta como mínimo dos tests; el conjunto propuesto usa exactamente dos.

2. Ejercicio 2: grafo de 7 nodos, *sidetrips* y PPC

- $N = \{1, 2, 3, 4, 5, 6, 7\}$, $N_0 = \{1\}$, $N_f = \{7\}$.
- $E = \{(1, 2), (1, 7), (2, 3), (2, 4), (3, 2), (4, 5), (4, 6), (5, 6), (6, 1)\}$.

Caminos candidatos del enunciado:

- $p_1 = [1, 2, 4, 5, 6, 1, 7]$
- $p_2 = [1, 2, 3, 2, 4, 6, 1, 7]$
- $p_3 = [1, 2, 3, 2, 4, 5, 6, 1, 7]$

2.1. (a) Requisitos para EPC

Los pares de aristas alcanzables son 12:

- $[1, 2, 3]$, $[1, 2, 4]$, $[2, 3, 2]$, $[2, 4, 5]$, $[2, 4, 6]$, $[3, 2, 3]$,
- $[3, 2, 4]$, $[4, 5, 6]$, $[4, 6, 1]$, $[5, 6, 1]$, $[6, 1, 2]$, $[6, 1, 7]$.

2.2. (b) ¿El conjunto candidato satisface EPC?

No. Pares cubiertos:

- p_1 : $[1, 2, 4], [2, 4, 5], [4, 5, 6], [5, 6, 1], [6, 1, 7]$.
- p_2 : $[1, 2, 3], [2, 3, 2], [3, 2, 4], [2, 4, 6], [4, 6, 1], [6, 1, 7]$.
- p_3 : $[1, 2, 3], [2, 3, 2], [3, 2, 4], [2, 4, 5], [4, 5, 6], [5, 6, 1], [6, 1, 7]$.

La unión cubre 10 de los 12 pares. Quedan sin cubrir $[3, 2, 3]$ y $[6, 1, 2]$, así que EPC no se satisface.

2.3. (c) Tour directo vs con *sidetrip*

Camino simple: $q = [3, 2, 4, 5, 6]$.

Camino de test: $t = [1, 2, 3, 2, 4, 6, 1, 2, 4, 5, 6, 1, 7]$.

t **no recorre q de manera directa**: cuando llega al nodo 4 por primera vez toma la rama $4 \rightarrow 6$ en lugar de $4 \rightarrow 5$, mientras que q exige $4 \rightarrow 5$.

t **sí recorre q mediante un *sidetrip***: desde el nodo 4 se desvía recorriendo $[4, 6, 1, 2, 4]$ y al volver al mismo nodo 4 retoma con $4 \rightarrow 5 \rightarrow 6$, completando la secuencia exigida por q . Como el desvío sale de 4 y vuelve a 4, encaja en la definición de *sidetrip*.

2.4. (d) Requisitos para NC, EC y PPC

NC: $\{1, 2, 3, 4, 5, 6, 7\}$.

EC: $(1, 2), (1, 7), (2, 3), (2, 4), (3, 2), (4, 5), (4, 6), (5, 6), (6, 1)$.

PPC (15 prime paths):

1. $[1, 2, 4, 6, 1]$
2. $[1, 2, 4, 5, 6, 1]$
3. $[2, 3, 2]$
4. $[2, 4, 6, 1, 2]$
5. $[2, 4, 5, 6, 1, 2]$
6. $[3, 2, 3]$
7. $[3, 2, 4, 6, 1, 7]$
8. $[3, 2, 4, 5, 6, 1, 7]$
9. $[4, 6, 1, 2, 3]$
10. $[4, 6, 1, 2, 4]$

11. [4, 5, 6, 1, 2, 3]
12. [4, 5, 6, 1, 2, 4]
13. [5, 6, 1, 2, 4, 5]
14. [6, 1, 2, 4, 6]
15. [6, 1, 2, 4, 5, 6]

2.5. (e) NC sin EC

Sí es posible. Por ejemplo:

$$t = [1, 2, 3, 2, 4, 5, 6, 1, 7].$$

Pasa por los siete nodos (cumple NC), pero al llegar a 4 toma la rama hacia 5 y nunca recorre el arco (4, 6). Como ese arco está en EC, EC no queda satisfecha.

2.6. (f) EC sin PPC

Sí es posible. Por ejemplo:

$$t_1 = [1, 2, 4, 5, 6, 1, 7], \quad t_2 = [1, 2, 3, 2, 4, 6, 1, 7].$$

La unión recorre los nueve arcos del grafo, así que EC se cumple. Pero ninguno de los dos pasa dos veces seguidas por el ciclo $2 \rightarrow 3 \rightarrow 2$, así que el prime path [3, 2, 3] queda sin cubrir y PPC no se satisface.

3. Ejercicio 3: CFG de `fmtRewrap`

El código se encuentra en el capítulo 7 del libro, el cual contiene el método `FmtRewrap.fmtRewrap(String S, int N)` que reemplaza los CR existentes y agrega CRs en los espacios más cercanos antes de la columna N ; dos CR seguidos se interpretan como *hard returns* y se conservan.

3.1. CFG por bloques

Para que el análisis sea manejable uso un CFG por bloques, no a nivel de instrucción individual. Los nodos:

- **W**: condición del `while`.
- **C**: clasificación del carácter actual (cadena de `if/else if` sobre `col`, `CR`, `' '` e `inWord`).

- **BW**: caso `betweenWord`.
- **LB**: caso `lineBreak`.
- **CR?**: caso `crFound`, que internamente toma una decisión.
- **CRh**: rama *hard-CR* (cuando `i+1 < len && next == CR`).
- **CRs**: rama *soft-CR* (rama `else` de la anterior).
- **IW**: caso `inWord` o por defecto.
- **INC**: incremento `i++`.
- **EXIT**: `S = new String(SArr) + CR; return.`

Aristas:

$$W \rightarrow C, W \rightarrow EXIT, C \rightarrow \{BW, LB, CR?, IW\}, CR? \rightarrow \{CRh, CRs\}, \\ \{BW, LB, CRh, CRs, IW\} \rightarrow INC, INC \rightarrow W.$$

3.2. (a) CFG del método

El CFG del método es exactamente el grafo descrito arriba.

3.3. (b) Test que sale del `while` sin entrar al cuerpo

Un caso válido: $t = (, 10)$. Con `S =` la condición del `while` evalúa $0 < 0$ (falso), así que el flujo toma directamente $W \rightarrow EXIT$ sin pasar por nodos intermedios.

3.4. (c) Requisitos para NC, EC y PPC

NC: $\{W, C, BW, LB, CR?, CRh, CRs, IW, INC, EXIT\}$.

EC (14 arcos): $(W, C), (W, EXIT), (C, BW), (C, LB), (C, CR?), (C, IW), (BW, INC), (LB, INC), (CR?, CRh), (CR?, CRs), (CRh, INC), (CRs, INC), (IW, INC), (INC, W)$.

PPC: el CFG admite 47 prime paths (el listado completo está en el README del ejercicio).

3.5. (d) Suite NC pero no EC

Suite en `FmtRerwrapNodeCoverageTest.java`:

```

1 @Test
2 public void coversBetweenWordAndLineBreakCase() {
3     String out = FmtRewrap.fmtRewrap(" a", 3);
4     assertNotNull(out);
5     assertTrue(out.endsWith("\n"));
6 }
7
8 @Test
9 public void coversCrFoundCasesAndInWordWithoutTrailingCr() {
10    String out = FmtRewrap.fmtRewrap("x\n\ny\nz", 100);
11    assertEquals("x\n\ny z\n", out);
12 }

```

JaCoCo sobre fmtRewrap: LINE 29/29, BRANCH 16/16.

Lo que pasó: con esta implementación particular y el nivel de modelado del CFG, una suite que cubre todos los nodos termina cubriendo también todos los arcos que mide la herramienta. No conseguí una suite que satisfaga NC sin satisfacer EC al mismo tiempo, porque muchas sentencias y ramas están tan acopladas que llegar al nodo implica recorrer también su arco saliente.

3.6. (e) Suite EC pero no PPC

Suite en `FmtRewrapEdgeButNotPrimePathCoverageTest.java`. Cinco casos elegidos para forzar cada rama del `switch`:

```

1 @Test public void whileCanExitWithoutEnteringBody() {
2     assertEquals("\n", FmtRewrap.fmtRewrap("", 10));
3 }
4 @Test public void coversBetweenWordAndLineBreakCase() {
5     String out = FmtRewrap.fmtRewrap(" a", 3);
6     assertTrue(out.endsWith("\n"));
7 }
8 @Test public void coversCrFoundHardReturnBranch() {
9     String out = FmtRewrap.fmtRewrap("x\n\ny", 100);
10    assertTrue(out.endsWith("\n"));
11 }
12 @Test public void coversCrFoundSoftReturnSecondOperandFalse() {
13    assertEquals("x y\n", FmtRewrap.fmtRewrap("x\ny", 100));
14 }
15 @Test public void coversCrFoundSoftReturnFirstOperandFalse() {
16    assertEquals("x \n", FmtRewrap.fmtRewrap("x\n", 100));

```

JaCoCo: LINE 29/29, BRANCH 16/16.

Por qué no cumple PPC: la suite no recorre el prime path $[BW, INC, W, C, IW]$. Cada vez que el flujo entra a BW, la siguiente iteración relevante del bucle pasa por LB en lugar de IW, así que esa secuencia particular no aparece nunca en los caminos ejecutados.

3.7. (f) Suite PPC con *Best Effort Touring*

Suite en `FmtRewrapPrimePathBestEffortCoverageTest.java`. JaCoCo: LINE 29/29, BRANCH 16/16.

Análisis de prime paths:

- Total: 47.
- Cubiertos directamente: 43.
- No cubiertos: 4. Todos **infactibles** por la semántica del método.

Los infactibles:

1. $[CR?, CRh, INC, W, C, LB]$
2. $[CR?, CRs, INC, W, C, CR?]$
3. $[CRs, INC, W, C, CR?, CRh]$
4. $[CRs, INC, W, C, CR?, CRs]$

Por qué son infactibles:

- Después de CRh el código fija `col = 1`. Para que en la siguiente iteración se entre directamente a LB haría falta tener un N tan chico que en la iteración previa ya se hubiera disparado `col >= N`, y entonces no se habría podido tomar CRh. Las dos condiciones son mutuamente excluyentes.
- La rama CRs se toma cuando el siguiente carácter inmediato **no** es CR. Eso impide que en la iteración siguiente se vuelva a entrar al caso CR?, porque ese caso requiere detectar un nuevo CR justo ahí.

Conclusión: la suite cumple PPC bajo *Best Effort Touring*, ya que cubre todos los prime paths factibles del CFG y los únicos 4 que quedan fuera son demostrablemente infactibles. Mantiene cobertura total de líneas y de ramas.

4. Ejercicio 4: instrumentación de patternIndex

4.1. Instrumentación

Para poder analizar los caminos ejecutados, agregué instrumentación a nivel de nodos del CFG dentro de `patternIndex()`. En cada invocación se registra la secuencia exacta de nodos por los que pasa la ejecución.

Nodos instrumentados:

- **S**: inicio del método.
- **W**: condición del `while` principal.
- **I**: `if` que compara el primer carácter del patrón.
- **M**: bloque donde hay match inicial y se inicializan las variables auxiliares.
- **FT**: condición verdadera del `for` interno.
- **IFM**: `if` de mismatch dentro del `for`.
- **MIS**: bloque de mismatch seguido de `break`.
- **FI**: paso de iteración del `for` cuando no hubo mismatch.
- **FF**: finalización del `for` sin haber ejecutado un `break`.
- **INC**: incremento `iSub++`.
- **R**: `return`.

El registro se delega a `PatternIndexPathTracker`, que va anotando los nodos `hit` y al final arma una `Invocation` con `subject`, `pattern`, resultado y la lista de nodos recorridos:

```
1 PatternIndexPathTracker.startInvocation(subject, pattern);
2 PatternIndexPathTracker.hit("S");
3
4 while (true){
5     PatternIndexPathTracker.hit("W");
6     if (!(isPat == false && iSub + patternLen - 1 < subjectLen)){
7         break;
8     }
9     PatternIndexPathTracker.hit("I");
10    if (subject.charAt(iSub) == pattern.charAt(0)){
11        PatternIndexPathTracker.hit("M");
12        // ...
13    }
```

```

14     PatternIndexPathTracker.hit("INC");
15     iSub++;
16 }
17 PatternIndexPathTracker.hit("R");
18 PatternIndexPathTracker.endInvocation(rtnIndex);

```

4.2. Suite

Suite en `PatternIndexInstrumentationCoverageTest.java`. Ejecuta los 10 casos de la tabla del enunciado:

subject	pattern	esperado
.a"	"bc"	-1
.ab"	.a"	0
.ab"	.ab"	0
.ab"	.ac"	-1
.ab"	"b"	1
.ab"	ç"	-1
.abc"	.abc"	0
.abc"	.abd"	-1
.abc"	"ba"	-1
.abc"	"bc"	1

Después de correr los casos, un `@AfterClass` genera un reporte con:

- la secuencia de nodos por cada caso,
- la cobertura de arcos sobre el CFG instrumentado,
- la cobertura de prime paths,
- la lista de requisitos cubiertos y faltantes.

El reporte queda en `target/patternindex-path-report.txt`.

4.3. Resultados de cobertura

- Cobertura de arcos: 15/15 (100%).
- Cobertura de prime paths: 22/39.

Interpretación:

- Los 10 casos de la tabla son suficientes para ejercitar todas las decisiones del método: EC al 100%.

- PPC no se completa, lo cual es esperable: PPC es estrictamente más fuerte que EC, así que una suite que satura EC no necesariamente cubre todos los prime paths. Los requisitos faltantes corresponden a combinaciones de iteraciones del bucle externo y del `for` interno que esos diez casos puntuales no llegan a ejercitar.

4.4. Cómo ejecutar

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local=.m2  
-Dtest=PatternIndexInstrumentationCoverageTest test
```